

Security Review — PrivacyPortal & PrivacyPortalFactory

coti-io/coti-contracts @ 8a0c4928004dac7a6c8d50bde58a022b6912f963 · PrivacyPortal & PrivacyPortalFactory · 2026-09-05 · rev 2 (PoC-validated, independently critiqued)

Target: `coti-io/coti-contracts` @ commit `8a0c4928004dac7a6c8d50bde58a022b6912f963` **Method:** `pashov/skills` `solidity-auditor` v3 (`c577eb7`) — 12 parallel specialist agents (math-precision, access-control, economic-security, execution-trace, invariant, periphery, first-principles, asymmetry, boundary, numerical-gap, trust-gap, flow-gap) on `opus` , followed by orchestrator dedup, four-gate judging, and lead promotion. **Date:** 2026-09-05 · **Revision:** v2 (PoC-validated + independently critiqued)

Scope

Repository / commit	<code>coti-io/coti-contracts</code> @ <code>8a0c4928004dac7a6c8d50bde58a022b6912f963</code>
Mode	filename
Files reviewed	<code>contracts/pod/privacy/PrivacyPortal.sol</code> · <code>contracts/pod/privacy/PrivacyPortalFactory.sol</code>
Confidence threshold (1-100)	80

Dedup / judging notes. 25 unique (Contract, function) tuples in the raw agent output; all 25 covered before gating. Rejected at gating: `PrivacyPortal.rescueNative` (admin-only fee redirect, no unprivileged amplifier), `PrivacyPortalFactory._revokeRole` (cosmetic), `PrivacyPortalFeeLib.packFeeConfig` naming (out of scope). Dropped as cosmetic/admin-adversarial: the `requestWithdrawWithPermit` event-field mismatch and the `setLimits` "maxWithdraw = 0" freeze.

Composite chain. `Chain: [10] + [1]` at confidence 75 — a full-balance `rescueERC20` during remount feeds the stuck-withdrawal precondition; combined, the user loses the pTokens *and* the collateral leaves to `rescueRecipient` .

Verification methodology (rev 2)

Revision 2 adds two independent layers on top of the rev 1 agent findings:

1. Executable PoCs (15 tests, all passing). Every finding was turned into a Hardhat test that drives the **unmodified** coti-contracts code at `8a0c4928` — `PrivacyPortal` , `PrivacyPortalFactory` , `PodErc20MintableInitializable` (→ `PodErc20Mintable` → `PodERC20`), `PrivacyPortalFeeLib` , `PortalFeeOracle` , and (for #5) `PodErc20CotiMother` — through constructor-passthrough harnesses that add no logic (`test/portal-poc/contracts/PortalHarnesses.sol`). The repo's own test mocks (`MockERC20` , `MockFeeOnTransferERC20` , `MockWrappedNative` , `RejectEthReceiver`) are reused unchanged. The only stand-in is the source-chain inbox (`MockInboxForPortal.sol`), which reproduces the real request-id packing and per-target nonce of `InboxBase` and delivers callbacks as `msg.sender == inbox` , so `InboxUser.onlyInboxPeer` /

`onlyInboxReturnLeg` and the real `transferCallback` / `transferError` / `invalidatePendingRequest` / `killStaleRequest` paths execute for real. Permits are real EIP-712 signatures verified by `PodERC20._consumePublicTransferPermit`.

```
export SOLC_NATIVE=/path/to/solc-static-linux # 0.8.28+commit.7893614a
NODE_OPTIONS='--max-old-space-size=8192' npx hardhat --config hardhat.config.portal-poc.ts test \
  test/portal-poc/findings-A.ts test/portal-poc/findings-B.ts
# → 15 passing (F1 has two variants; F2–F14 one each)
```

2. Independent adversarial critique. A separate reviewer session with no access to the rev 1 agents' reasoning re-ran the suite, re-read the audited code, and for each finding judged whether the PoC exercises the real claim, whether the precondition is realistic (who must act), whether the behaviour is a documented design choice, whether existing mechanisms already mitigate it, and whether the proposed fixes are implementable. Verdict vocabulary: **CONFIRMED** (finding stands as written), **CONFIRMED-DOWNGRADED** (real, but lower severity than rev 1 implied), **DESIGN-CHOICE** (real behaviour, documented/accepted, not a vulnerability), **DUPLICATE**, **NOT-REPRODUCED**. The "Final severity" column below is the reconciled result of the PoC evidence and the critique; where the critic and rev 1 disagreed, the critic's reasoning is quoted in the finding.

Outcome. PoC suite: **15 passing / 0 failing** (the critic re-ran it: same result, no files modified; `contracts/` tree verified byte-identical to `8a0c4928`). Verdicts: **CONFIRMED** 5 · **CONFIRMED-DOWNGRADED** 6 · **DESIGN-CHOICE** 2 · **DUPLICATE** 1 · **NOT-REPRODUCED** 0. Final severities: **Medium** 3 (#1, #4, #14) · **Low** 8 · **Informational** 3 · no Critical/High. The rev 1 confidence scores (which measured agent convergence, not exploitability) are retained for reference; the **Final severity** column supersedes them. Fix-quality issues found during verification: #3 Fix B is unimplementable, #1 Fix A covers only the native case, #4's fix should also retire the previous portal (details in each finding and in *Corrections*).

Findings

[90] **1. Withdrawal whose pToken transfer succeeded but whose payout reverts is permanently stuck — both legs frozen, pTokens unburnable**

`PrivacyPortal._releaseWithdrawal` · Confidence: 90 · [agents: 9]

Description Once `pToken.requests(transferRequestId).status == Success`, a payout that reverts (a non-payable recipient on a native-wrapped portal, or an issuer-blacklisted recipient on an ERC-20 portal) makes `_releaseWithdrawal` revert forever while `cancelFailedWithdrawal` is unreachable (it requires `Failed` / `SystemFailed`), so the user's pTokens sit in portal custody outside `pendingBurnAmount` (never burnable) and the collateral is never released.

Fix (Option A — allow-and-handle)

```
IWrappedNative(address(underlyingToken)).withdraw(withdrawal.amount);
(bool ok,) = payable(withdrawal.recipient).call{value: withdrawal.amount}("");
- if (!ok) {
-     revert EthTransferFailed();
- }
+ if (!ok) {
+     // do not revert: re-wrap and deliver the ERC-20, or credit a pull-payment claim
+     IWrappedNative(address(underlyingToken)).deposit{value: withdrawal.amount}();
+     underlyingToken.safeTransfer(withdrawal.recipient, withdrawal.amount);
+ }
```

Fix (Option B — admin re-target / terminalize)

```

+ /// Admin-gated, whenPaused counterpart to adminRefundPendingDeposit for a TransferPending
+ /// withdrawal whose transferRequestId is already Success: re-target withdrawal.recipient,
+ /// or terminalize it while crediting pendingBurnAmount so custodied pTokens stay burnable.
+ function adminResolveStuckWithdrawal(bytes32 withdrawalId, address newRecipient)
+     external onlyFactoryAdmin whenPaused nonReentrant { ... }

```

Verification result — PoC: ✓ F1a native · ✓ F1b ERC-20 · Independent verdict: **CONFIRMED** · Final severity: **Medium** (confidence 92)

PoC — `test/portal-poc/findings-A.ts` > F1a (native-wrapped portal, recipient = `RejectEthReceiver`) and F1b (ERC-20 portal, recipient blocked by the token issuer **after** the request). Real `PodERC20.transferCallback` marks the transfer `Success`, then the portal hook `onPTokenTransferred` → `_releaseWithdrawal` reverts and the pToken swallows it (`RequestCallbackFailed`). Observed on the real contracts:

Step	Result
<code>pToken.requests(transferRequestId).status</code>	<code>Success</code> (terminal)
<code>withdrawals[id].status</code>	<code>TransferPending</code> (unchanged)
<code>triggerWithdrawalRelease</code> (anyone)	reverts <code>EthTransferFailed</code> (F1a) / <code>IssuerBlocked</code> (F1b), every time
<code>cancelFailedWithdrawal</code>	reverts <code>WithdrawTransferNotFailed</code>
<code>factory.killPTokenStaleRequest</code> after 2 days	reverts <code>RequestNotPending</code> (status is <code>Success</code>)
<code>pendingBurnAmount</code> / <code>burnAccumulatedPTokens</code>	<code>0</code> / reverts <code>PendingBurnTooLow</code> — custodied pTokens are uncounted
ABI entry points taking <code>withdrawalId</code>	only <code>onPTokenTransferred</code> , <code>triggerWithdrawalRelease</code> , <code>cancelFailedWithdrawal</code> — no re-target
<code>rescueERC20</code> (admin, paused)	moves the collateral to <code>rescueRecipient</code> , not to the user

Independent verification (critic)

CONFIRMED · **Medium** · confidence 92. The PoC drives the release through the genuine pToken `transferCallback` hook (`PodERC20.sol:366-373`); the terminal-status wall is the real `_setRequestStatus` guard (`PodERC20.sol:630-637`) plus the status checks in `_releaseWithdrawal` (`PrivacyPortal.sol:862-864`) and `cancelFailedWithdrawal` (:655-660) — nothing in the harness manufactures it. Precondition is unprivileged: F1a is user error (recipient cannot receive ETH); F1b is the realistic trigger (a token issuer blocklists the recipient mid-flight). Impact: pTokens already moved into portal custody on COTI, collateral locked, `pendingBurnAmount` never incremented (the `+=` at :866 is rolled back with the revert), no re-target function; recovery is admin-only and pays `rescueRecipient`, not the user. **Fix critique:** Option B (`adminResolveStuckWithdrawal`) is correct; **Option A only fixes F1a** — re-wrapping to WETH does nothing when the ERC-20 transfer itself reverts (F1b).

[85] 2. Blacklist (and pause) are enforced only at request time; the collateral-release leg pays blacklisted recipients from a paused portal

`PrivacyPortal._releaseWithdrawal` · Confidence: 85 · [agents: 6]

Description `_releaseWithdrawal` — reachable permissionlessly via `triggerWithdrawalRelease` and via the attacker-forgeable `onPTokenTransferred` callback — checks neither the per-portal/factory blacklist nor `paused()`, so a withdrawal requested before a blacklisting or pause still pays out afterwards, defeating the

control the code's own comment promises and letting collateral leave a paused portal during the `whenPaused` rescue/migration window.

Fix (Option A — validate blacklist)

```
if (requestStatus != IPodERC20.RequestStatus.Success) {
    revert PTokenTransferNotSuccessful(withdrawal.transferRequestId, requestStatus);
}
+ _checkNotBlacklistedAccount(withdrawal.recipient); // read the factory list via bindingFactory
+ _checkNotBlacklistedAccount(withdrawal.user);
```

Fix (Option B — validate blacklist + pause via bindingFactory)

```
+ if (paused() || _controllerFactory().withdrawalsPaused()) revert WithdrawalsPaused();
+ if (blacklisted[withdrawal.recipient] ||
_controllerFactory().blacklisted(withdrawal.recipient)) {
+     revert AddressBlacklisted(withdrawal.recipient);
+ }
```

Either option must be paired with an admin path to re-target/refund a withdrawal that becomes unreleasable — otherwise it converts into finding #1.

Verification result — PoC: ✓ F2 · Independent verdict: **CONFIRMED-DOWNGRADED** · Final severity: **Low** (confidence 85)

PoC — `findings-A.ts` > F2. Two withdrawals to `user2` are requested while `user2` is clean. Admin then blacklists `user2` on the factory **and** the portal, and pauses both the portal instance and the factory. New requests naming `user2` revert `AddressBlacklisted`; any new request reverts `WithdrawalsPaused`. Yet:

- the COTI success callback releases withdrawal #1 to the blacklisted `user2` while everything is paused (`callbackFailed == false`, status `Released`);
- withdrawal #2 (its hook made to fail once by the documented pause → rescue step, then the portal re-funded) is released by a **stranger** via `triggerWithdrawalRelease` — again to the blacklisted recipient, again while paused.

`paused() == true` and `factory.blacklisted(user2) == true` hold at both payouts. `_releaseWithdrawal` (PrivacyPortal.sol:853-886) contains no pause or blacklist check.

Independent verification (critic)

CONFIRMED-DOWNGRADED · **Low** · confidence 85. Both halves reproduce on the real contracts, but the **pause half is intended behaviour, not a bug**: the repo's own test

`PrivacyPortalFactory.portalRemount.test.ts:383-435` ("completes in-flight TransferPending withdraw on old portal after remount") requires `triggerWithdrawalRelease` to work on a paused, retired portal, and the migration model at PrivacyPortalFactory.sol:600-601 depends on it; adding the report's Option B pause check would turn every in-flight withdrawal at pause time into finding #1. The blacklist half is real but narrow: only withdrawals requested *before* the listing slip through; request-time checks (PrivacyPortal.sol:519, 1031-1035) hold for everything else, and the code comment at :1028-1030 only promises recipient checking at request time. Impact is a compliance window equal to the in-flight set — no collateral loss, no unbacked supply. The PoC's second leg (rescue → re-fund → stranger trigger) is theatrical; leg (a) alone proves the claim.

[85] **3. Remount rotates the pToken minter away, bricking the retired portal's `adminRefundPendingDeposit` for every still-Pending mint**

`PrivacyPortal.adminRefundPendingDeposit` · Confidence: 85 · [agents: 9]

Description `adminRefundPendingDeposit` unconditionally calls `pToken.invalidatePendingRequest` (minter-gated by `_checkMinter`) whenever the mint is Pending, but `createPortalWithExistingPToken` / `setPTokenMinter` move `minter` to the new portal while every escrow stays on the old one, so the refund reverts `OnlyMinter(oldPortal)` and the permissionless `refundFailedDeposit` cannot substitute (it needs `SystemFailed`), leaving depositor collateral unrefundable until an undocumented minter-swap or a `requestKillMinAge`-gated kill.

Fix (Option A — authorize the owner/factory to invalidate)

```
// PodERC20.invalidatePendingRequest
- _checkMinter();
+ if (msg.sender != minter && msg.sender != owner()) revert OnlyMinter(msg.sender);
+ // plus a PrivacyPortalFactory forwarder (guarded by _requireFactoryOwnedPToken) that
+ // adminRefundPendingDeposit falls back to when pToken.minter() != address(this)
```

Fix (Option B — refuse remount while escrows are pending)

```
// PrivacyPortalFactory.createPortalWithExistingPToken, before setMinter
+ if (IPrivacyPortal(oldPortal).pendingEscrowCount() != 0) revert
  OldPortalHasPendingEscrows(oldPortal);
```

Fix (Option C — remove the kill delay before rotating)

```
+ IPodERC20(existingPToken).setRequestKillMinAge(0); // factory is the pToken's Ownable owner
  PodErc20Mintable(payable(existingPToken)).setMinter(portal);
```

Verification result — PoC: ✓ F3 (+3 recovery paths) · Independent verdict: **CONFIRMED-DOWNGRADED** · Final severity: **Low** (confidence 88)

PoC — `findings-A.ts` > F3. Deposit on portal A (mint `Pending`, escrow `Pending`) → `pause()` → `createPortalWithExistingPToken` (the documented remount). Then on the retired portal:

- `adminRefundPendingDeposit(requestId)` reverts `OnlyMinter(0x...oldPortal)` — raised by `PodERC20.invalidatePendingRequest` → `_checkMinter` (PodERC20.sol:573-574, PodErc20Mintable.sol:47-51);
- `refundFailedDeposit` reverts `DepositMintNotFailed` (status `Pending`, not `SystemFailed`);
- the new portal reverts `DepositEscrowInvalid` (it never saw the escrow).

Three admin-only recoveries were exercised from the same snapshot: (A) `killPTokenStaleRequest` reverts `RequestNotAged` until `requestKillMinAge` (1 day) elapses, then the refund succeeds; (B) `setPTokenMinter(pToken, oldPortal)` swap-back, refund, swap-forward; (C) `setPTokenRequestKillMinAge(pToken, 0)` then kill. None is part of the documented refund flow.

Independent verification (critic)

CONFIRMED-DOWNGRADED · **Low** · confidence 88. The revert is real and exactly as described (PrivacyPortal.sol:632-634 → PodERC20.sol:573-574 → PodErc20Mintable.sol:47-51; minter moved at PrivacyPortalFactory.sol:674); auth and `whenPaused` still pass via `bindingFactory` (:986-992), so the minter gate is the only blocker. But the PoC itself demonstrates three shipped, admin-only recovery paths — kill after `requestKillMinAge` (PodERC20.sol:67, 590-605), `setPTokenMinter` swap-back (PrivacyPortalFactory.sol:394-397), or `setPTokenRequestKillMinAge(0)` + immediate kill — and `adminRefundPendingDeposit` was DEFAULT_ADMIN-only to begin with. Ops speed bump, not unrecoverable depositor loss; the repo only tests the `SystemFailed` variant of this path (portalRemount.test.ts:235-272), which is why the gap survived. **Fix critique:** Option A and C work; **Option B cannot be implemented** — `pendingEscrowCount()` does not exist and cannot be derived, because escrows are only ever written `Pending` (:420, :468) or `Refunded` (:597, :638); a mint

success never terminalizes one, so such a counter would be permanently non-zero and would block every future remount.

[80] 4. Cross-factory remount skips every old-portal safety guard and rotates the minter away from a still-live portal

PrivacyPortalFactory.createPortalWithExistingPToken · Confidence: 80 · [agents: 4]

Description The OldPortalNotPaused, native-mode, and retireDepositsForUpgrade checks are nested under if (oldPortal != address(0)) where oldPortal is read from this factory's own portalForPToken, so on the documented transferPTokenOwnership → remount-on-new-factory path they are all skipped while setMinter(newPortal) still runs — leaving the previous portal unpaused, un-retired, still serving withdrawals against its own collateral, with one pToken supply floating over two disjoint pools.

Fix

```
+ address prevMinter = PodErc20Mintable(payable(existingPToken)).minter();
+ if (prevMinter != address(0) && prevMinter != portal && prevMinter.code.length != 0) {
+   if (!IPrivacyPortal(prevMinter).paused()) revert OldPortalNotPaused(prevMinter);
+   // apply the same nativeWrappedUnderlying equality check against prevMinter
+ }
PodErc20Mintable(payable(existingPToken)).setMinter(portal);
```

Verification result — PoC: ✓ F4 · Independent verdict: CONFIRMED · Final severity: Medium (confidence 88)

PoC — findings-A.ts > F4. Factory F1 creates portal A + pToken; user deposits (mint Success). A second real factory F2 (different admin) is deployed; F1's admin calls transferPTokenOwnership(pToken, F2); F2's admin calls createPortalWithExistingPToken while A is unpaused. No revert. Observed:

Check	Value
pToken.minter()	portal B (F2's clone)
A.paused() / A.isDepositEnabled() / A.factory()	false / true / F1 — none of the same-factory guards ran
deposit on A	reverts OnlyMinter(A) deep inside the pToken, although A advertises "open"
requestWithdrawWithPermit on A + COTI success	Released — A keeps paying withdrawals from its own collateral
A.pause() by F2's admin	reverts OnlyFactoryAdmin (A's auth is bindingFactory == F1)
B unpaused by F2's admin, withdrawal on B	accepted; release reverts ERC20InsufficientBalance (B holds no collateral) → F1-shaped stuck withdrawal

Independent verification (critic)

CONFIRMED · Medium · confidence 88. The strongest PoC of the set: a second real factory with a different admin, the real transferPTokenOwnership, then createPortalWithExistingPToken — ending with one pToken supply over two disjoint collateral pools. Root cause confirmed: oldPortal is read from this factory's portalForPToken (PrivacyPortalFactory.sol:622, 651), so on the new factory it is address(0) and the whole guard block at :652-666 is skipped while setMinter at :674 still runs. Precondition is two DEFAULT_ADMIN actions — but the repo's own test PrivacyPortalFactory.createPortalWithExistingPToken.test.ts:84-124 performs exactly this hand-off and remount on an unpaused portal, and :411 documents "after handoff, remount on the new factory" as supported, so it is a blessed path whose safety rail is silently absent. Impact: portal A

reports open while deposits revert `OnlyMinter`, keeps paying out its collateral, and cannot be paused by the new factory's admin; B is openable with zero collateral, where withdrawals become finding #1. **Fix critique:** reading authority from the token (`prevMinter`) is the right shape; the diff should also call `retireDepositsForUpgrade()` on `prevMinter`.

[80] 5. `createPortal` returns a live, deposit-enabled portal before the one-way COTI registration has landed

`PrivacyPortalFactory.createPortal` · Confidence: 80 · [agents: 3]

Description Unlike `createPortalWithExistingPToken` (which calls `pauseByFactory()`), `createPortal` never pauses the clone and `initialize` hardcodes `isDepositEnabled = true`, so deposits made before the fire-and-forget `registerToken` message executes on COTI hit `TokenNotRegistered` (a retryable revert that leaves the mint `Pending`) and lock collateral behind an admin-only, pause-gated refund.

Fix (Option A — start closed)

```
IPrivacyPortal(portal).initialize(underlying, pToken, decimals, nativeWrappedUnderlying,
address(this));
+ IPrivacyPortal(portal).pauseByFactory(); // admin unpauses after observing TokenRegistered on
COTI
```

Fix (Option B — safe default in initialize)

```
// PrivacyPortal.initialize
- isDepositEnabled = true;
+ isDepositEnabled = false;
```

Verification result — PoC: ✓ F5 (incl. real mother) · Independent verdict: **DESIGN-CHOICE** · Final severity: **Low** (confidence 90)

PoC — `findings-A.ts` > F5. Immediately after `createPortal`: `paused() == false`, `isDepositEnabled == true`. The registration request recorded by the inbox is one-way (`twoWay == false`, `errorSelector == 0x00000000`) targeting the mother. A user deposit is accepted (collateral locked, mint `Pending`); `refundFailedDeposit` reverts `DepositMintNotFailed`; `adminRefundPendingDeposit` reverts `ExpectedPause`. COTI half on the real `PodErc20CotiMother`: a `mintPublic` presented as the (allow-listed factory's) pToken before `registerToken` reverts `TokenNotRegistered(0x5d0028cf)`; after `registerToken` the same call passes the registration gate. The factory NatSpec (`PrivacyPortalFactory.sol:531-542`) documents exactly this window ("PP-02/PP-14") and asks operators to keep `isDepositEnabled` off manually.

Independent verification (critic)

DESIGN-CHOICE · **Low** · confidence 90. Every sub-claim reproduces, end-to-end including the real mother (`TokenNotRegistered` before `registerToken`, gate passed after; `PodErc20CotiMother.sol:113-120, 317`). But the behaviour is **documented verbatim** at `PrivacyPortalFactory.sol:530-542`: the function "returns as soon as the message is *submitted*", mints will hit `TokenNotRegistered` and leave deposits "stuck `Pending` rather than failing cleanly (see PP-02/PP-14)", and operators are told to set `isDepositEnabled = false` immediately. The residual is real but small: the guidance is not atomically enforceable — between `createPortal` (`DEPLOYER_ROLE`, the lowest-trust role, :549) and the admin's follow-up there is a window in which any user can lock collateral behind a pause-gated admin refund. Minor doc inaccuracy: NatSpec says "leave ... `false`" but `initialize` hardcodes `true` (`PrivacyPortal.sol:255`). Fix Option A (`pauseByFactory()` in `createPortal`) is the cleaner one; `unpause()` still works because `factory != 0` (:272-277).

[80] 6. Minter rotation in the remount path inspects none of the retiring portal's in-flight escrows (factory side of #3)

`PrivacyPortalFactory.createPortalWithExistingPToken` · Confidence: 80 · [agents: 2]

Description `setMinter(portal)` is executed unconditionally in the remount path with no check of the retiring portal's Pending deposit escrows, which is precisely what revokes the authority finding #3 depends on — and the test mocks (`MockPodErc20MintableForPortal` lacks `invalidatePendingRequest`; `MockPodERC20ForPortal`'s has no minter gate) cannot observe it.

Fix (Option A — keep the retired portal authorized)

```
+ // let the pToken's Ownable owner (the factory) also authorize invalidatePendingRequest,  
+ // and add a factory forwarder so a retired portal's admin refund can terminalize the mint
```

Fix (Option B — drain before rotating)

```
+ // require the old portal to report zero Pending escrows (or invalidate them via a  
+ // factory-mediated call) before PodErc20Mintable(existingPToken).setMinter(portal)
```

Verification result — PoC: ✓ F6 · Independent verdict: **DUPLICATE** · Final severity: **Informational** (confidence 95)

PoC — `findings-A.ts` > F6. With a `Pending` escrow on portal A, `createPortalWithExistingPToken` completes and rotates the minter — the factory reads nothing from the retiring portal (`PrivacyPortalFactory.sol:651-674`). The shipped mocks cannot observe the consequence: a raw call to `invalidatePendingRequest(bytes32)` on `MockPodErc20MintableForPortal` reverts (function absent), and `MockPodERC20ForPortal.invalidatePendingRequest` is callable by a stranger (no minter gate). Note: escrows are never marked terminal on mint `Success` either (only refunds change `DepositEscrowStatus`), so a "pending escrow count" does not exist on-chain today.

Independent verification (critic)

DUPLICATE of #3 · **Informational** · confidence 95. Same root cause, precondition, impact and fix menu as #3 (the report's own title says "factory side of #3"); it should have been merged at the dedup stage rather than scored separately. The genuinely new content — the shipped mocks cannot exercise the minter gate (`MockPodErc20MintableForPortal` has no `invalidatePendingRequest`; `MockPodERC20ForPortal.sol:197` has no minter gate) — is a valid **test-coverage** observation. Useful corollary surfaced by the PoC and missing from the report: escrows are never terminalized on mint success, which is exactly what breaks #3's Fix Option B.

[75] 7. Operator role can freeze all withdrawals via an unbounded `fixedFee` floor

`PrivacyPortal.setWithdrawFee` (and `PrivacyPortalFactory.setDefaultWithdrawFee`) · Confidence: 75 · [agents: 2]

Description `packFeeConfig` caps `percentageBps` at 10% but bounds `fixedFee` only by `uint96`, so an `OPERATOR_ROLE` holder — documented for "routine fee-parameter updates" — can set a withdraw-fee floor no wallet can pay (per portal, or factory-wide in one call) and block every `requestWithdrawWithPermit` with no `Paused` event, `paused()==false`, and no rescue enablement, an emergency-stop the role model reserves for `DEFAULT_ADMIN_ROLE`.

Verification result — PoC: ✓ F7 · Independent verdict: **CONFIRMED** · Final severity: **Low** (confidence 88)

PoC — `findings-A.ts` > F7. `operator` holds only `OPERATOR_ROLE`: `portal.pause()` reverts `OnlyFactoryAdmin`, `factory.pause()` reverts `AccessControlUnauthorizedAccount`.

`portal.setWithdrawFee(2^96-1, 0, 2^96-1)` is accepted (`packFeeConfig` bounds `fixedFee` by `uint96` only, `PrivacyPortalFeeLib.sol:29-34`); every withdrawal then reverts `InsufficientPortalFee(79228162514264337593543950335, 0)` while `paused() == false` and `rescueERC20` still reverts `ExpectedPause`. `factory.setDefaultWithdrawFee` (also `OPERATOR_ROLE`) reproduces it factory-wide in one call; lowering the fee restores withdrawals.

Independent verification (critic)

CONFIRMED · **Low** · confidence 88. Clean PoC with a separately-granted operator: cannot pause (`portal` `OnlyFactoryAdmin`, `factory` `AccessControlUnauthorizedAccount`), can set `fixedFee = 2^96-1` per portal (`PrivacyPortal.sol:363-367`) or factory-wide (`PrivacyPortalFactory.sol:463-470`); `packFeeConfig` caps `percentageBps` at 10% but `fixedFee` only by `uint96` (`PrivacyPortalFeeLib.sol:29-34`) and `_portalFeeFloor` returns it unmodified when `bps == 0` (`PrivacyPortal.sol:909-911`). A genuine privilege-boundary violation: `OPERATOR_ROLE` is NatSpec'd for "routine fee-parameter updates" (`PrivacyPortalFactory.sol:32`) while `pause()` is reserved for admin (:433; `PrivacyPortal.sol:266`). Impact is a silent DoS on new deposits/withdrawals (no `Paused` event, `paused()` false for monitors); rescue stays locked behind `whenPaused`, already-`TransferPending` withdrawals still release, and any operator/admin restores service in one transaction. No fix was proposed: add an absolute `fixedFee` ceiling, or make `fixedFee` admin-only while leaving `percentageBps` to operators.

[75] 8. `refundFailedDeposit` rejects mint status `Failed`, which the protocol's own ops tools write

`PrivacyPortal.refundFailedDeposit` · Confidence: 75 · [agents: 4]

Description The permissionless refund accepts only `SystemFailed`, but `killPTokenStaleRequest` / `invalidatePendingRequest` terminalize a stuck mint as `Failed`, so clearing a stale mint kills the permissionless refund and forces a full-portal pause plus admin refund for a single depositor — unlike `cancelFailedWithdrawal`, which accepts both terminal states (and `DepositEscrowStatus.Failed` is never written anywhere, leaving that branch dead).

Verification result — PoC: ✓ F8 · Independent verdict: **CONFIRMED-DOWNGRADED** · Final severity: **Low** (confidence 85)

PoC — `findings-B.ts` > F8. `setPTokenRequestKillMinAge(0)` + `killPTokenStaleRequest` terminalize a stuck mint as `Failed` (`PodERC20.sol:602`). `refundFailedDeposit` then reverts `DepositMintNotFailed(id, Failed)` (`PrivacyPortal.sol:592-594`); `adminRefundPendingDeposit` reverts `ExpectedPause` until the whole portal is paused, then succeeds. Contrasts exercised in the same test: a `SystemFailed` mint is refundable by a stranger; `cancelFailedWithdrawal` accepts a killed (`Failed`) transfer (`PrivacyPortal.sol:655-660`).

Independent verification (critic)

CONFIRMED-DOWNGRADED · **Low** · confidence 85. Technically flawless PoC; the dead `DepositEscrowStatus.Failed` branch also checks out (accepted at :586, never written). **The report's framing is wrong**: "clearing a stale mint *kills* the permissionless refund" — before the kill the mint is `Pending`, for which `refundFailedDeposit` reverts as well (the PoCs show this at findings-A.ts:151 and :254), so no capability is destroyed; the kill in fact *enables* `adminRefundPendingDeposit` by ruling out a late `Success`. The exclusion of `Failed` is documented at `IPrivacyPortal.sol:155-159` ("App `raise` / `Failed` is not refundable"). Residual: the asymmetry with `cancelFailedWithdrawal` is a real design inconsistency and the operational cost is real (pausing the whole portal to refund one depositor, :619). A safe improvement is to accept `Failed` only when the portal itself invalidated/killed the request.

[75] 9. Deposit limits and the fee are checked on the requested `amount` , but the position is minted from the measured `received`

`PrivacyPortal._deposit` (and `depositNative`) · Confidence: 75 · [agents: 8]

Description `_checkDepositLimits(amount)` runs before the balance-delta measurement while escrow and mint use `received` , so with a fee-on-transfer underlying (an explicitly supported case, with a 5% mock in the repo) a deposit that passes `minDepositAmount` creates a position below `minWithdrawAmount` that can never be withdrawn, and the percentage fee is charged on the pre-fee notional.

Verification result — PoC: ✓ F9 · Independent verdict: **CONFIRMED-DOWNGRADED** · Final severity: **Low** (confidence 82)

PoC — `findings-B.ts` > F9 with the repo's own `MockFeeOnTransferERC20` (5%) and `setLimits(min=100, ..., minWithdraw=100, ...)` . Deposit of 100 passes `_checkDepositLimits(100)` ; escrow/mint carry the measured 95 (`depositEscrows[id].amount == 95e6`). Withdrawals of 95 and 99 revert `WithdrawBelowMinimum` ; a second deposit makes 190, a 100 withdrawal succeeds and the 90 residue is again unredeemable. Fee side with the real `PortalFeeOracle` at 1%: `getDepositPortalFeeFloor(100) == 1.00 native` vs `0.95 native` for 95; a deposit declaring the measured-amount fee reverts `InsufficientPortalFee` .

Independent verification (critic)

CONFIRMED-DOWNGRADED · **Low** · confidence 82. Mechanism exactly as stated: `_checkDepositLimits(amount)` (:404) and `_validateAndCollectPortalFee(portalFee, amount, true)` (:406) precede the balance-delta measurement (:412-414) while escrow and mint use `received` (:419-425); the fee-floor asymmetry is shown against the real factory view. Precondition needs a fee-on-transfer underlying (explicitly supported, "PP-06") and an admin setting `minWithdraw ≈ minDeposit` with no headroom; under the defaults (`minDeposit = minWithdraw = 1` , :258-259) it cannot occur. Impact partly overstated: the first half (one deposit at exactly `minDeposit` → unwithdrawable position when top-ups are impossible) stands; the PoC's second half is self-inflicted — a holder of 190 should withdraw 190 in one call rather than 100 then report the 90 residue. Fee overcharge is real but ≤ the transfer-tax rate. No fix proposed; the correct one is to run the limit/fee checks after measurement and charge on `received` .

[75] 10. `rescueERC20` has no reserve for withdrawals already in `TransferPending` , so the documented migration drains committed collateral

`PrivacyPortal.rescueERC20` · Confidence: 75 · [agents: 4]

Description The portal keeps no aggregate of in-flight withdrawals and `rescueERC20` excludes only the pToken, so the pause → remount → rescue-full-balance procedure (exactly what the repo's tests perform) removes collateral that pending withdrawals still need; when those transfers settle `Success` the release reverts on balance, the pTokens are custodied but uncounted, and the user ends with neither leg.

Verification result — PoC: ✓ F10 · Independent verdict: **CONFIRMED-DOWNGRADED** · Final severity: **Low** (confidence 87)

PoC — `findings-B.ts` > F10. Withdrawal `TransferPending` → `pause` → remount → `rescueERC20(underlying, fullBalance)` succeeds (no reserve; `PrivacyPortal.sol:769-782`) → COTI success callback: hook fails, transfer `Success` , `triggerWithdrawalRelease` reverts `ERC20InsufficientBalance` , `cancelFailedWithdrawal` reverts `WithdrawTransferNotFailed` , `pendingBurnAmount == 0` . Recovery: only an out-of-band ERC-20 transfer back to the retired portal, after which `triggerWithdrawalRelease` succeeds.

Independent verification (critic)

CONFIRMED-DOWNGRADED · **Low** · confidence 87. The documented migration (pause → remount → rescue full balance) strands a real in-flight withdrawal; `rescueERC20` (:769-782) excludes only the pToken and consults no

obligation total because none exists — `pendingBurnAmount` is incremented *inside* the release (:866) and never reflects committed-but-unreleased withdrawals. Not contrived: the repo's migration test rescues the full balance (portalRemount.test.ts:147-156) while another repo test (:383-435) relies on in-flight withdrawals settling against that same balance — the PoC is the intersection. But recovery is a plain ERC-20 transfer back to the retired portal, after which anyone can release (PoC), so no permanent loss unless `rescueRecipient` is unreachable. The real finding is the missing on-chain obligation counter. No fix proposed; track an outstanding-withdrawal total (incremented at request, decremented at release/cancel) and floor `rescueERC20` on `underlyingToken` at it.

[75] 11. Fee floor silently collapses to `fixedFee` (legally 0) when the oracle returns a zero rate

`PrivacyPortal._portalFeeFloor` · Confidence: 75 · [agents: 4]

Description `resolvePortalFee` returns `(fixedFee, false)` on any zero rate and `_portalFeeFloor` discards the `usedDynamicPricing` flag, so a portal whose underlying was never priced (`createPortal` sets no peg), a cleared peg, or a stale-to-zero adapter degrades a configured percentage fee to the flat fee — which `packFeeConfig` allows to be 0 — with no revert and no event.

Verification result — PoC: ✓ F11 · Independent verdict: **DESIGN-CHOICE** · Final severity: **Informational** (confidence 90)

PoC — `findings-B.ts` > F11 with the real `PortalFeeOracle`. Priced (1 USD each, 1% fee): a withdrawal with `portalFee = 0` reverts `InsufficientPortalFee(1e18, 0)`. After `clearTokenPriceUSD(underlying)` the same call succeeds with fee 0 (`estimateWithdrawFees` → `(0, usedDynamicPricing=false)`; the tx path discards that flag, PrivacyPortal.sol:917). A second portal created for a never-priced underlying under a 1% default deposit fee accepts a deposit at fee 0 (`accumulatedPortalFees == 0`). `IPodPriceOracle.sol:178-179` and `PortalFeeOracle.sol:107` document zero rates as "fixed fee only".

Independent verification (critic)

DESIGN-CHOICE · **Informational** · confidence 90. Reproduces precisely against the real `PortalFeeOracle`, but fail-open is **documented three times**: `IPodPriceOracle.sol:9-10` ("adapters never revert; return 0 when a feed is unset, stale, or failed. Zero rates make resolvePortalFee skip dynamic pricing (fixed fee only)"), `PortalFeeOracle.sol:9`, and `resolvePortalFee`'s own `@dev` (PrivacyPortalFeeLib.sol:78). The alternative — reverting on a zero rate — would brick deposits *and the exit path* factory-wide on any feed outage, which the report's own leads section identifies as a hazard. Impact is protocol fee revenue only; detection exists (`estimateDepositFees` / `estimateWithdrawFees` return `usedDynamicPricing`, :785-812; `PortalFeeOracle.tokenPriceUpdatedAt` / `getTokenPriceMeta` for staleness alarms).

[75] 12. `setLimits` accepts `maxWithdraw < 2·minWithdraw`, leaving balances in the gap unredeemable

`PrivacyPortal.setLimits` · Confidence: 75 · [agents: 1]

Description Each min/max pair is validated only against itself, so with `maxWithdraw < 2·minWithdraw - 1` a balance in `(maxWithdraw, 2·minWithdraw)` cannot be split into any legal sequence of withdrawals and its sub-minimum residue is permanently locked whenever deposits are disabled or the portal is retired.

Verification result — PoC: ✓ F12 · Independent verdict: **CONFIRMED-DOWNGRADED** · Final severity: **Informational** (confidence 80)

PoC — `findings-B.ts` > F12. `setLimits(1, max, 100, 150)` is accepted (PrivacyPortal.sol:342-347 validate each pair only against itself). With a 170 balance: 170 → `WithdrawExceedsMaximum`; 20, 70, 99 → `WithdrawBelowMinimum`; no two legal parts sum to 170. `setIsDepositEnabled(false)` (operator) then removes the only top-up route (`DepositDisabled`).

Independent verification (critic)

CONFIRMED-DOWNGRADED · **Informational** · confidence 80. Arithmetic correct and `setLimits` does accept it (:342-347), but it is a pure `DEFAULT_ADMIN` misconfiguration nothing unprivileged can trigger. The stranded residue is bounded by `minWithdraw - 1` per holder (withdraw `maxWithdraw` first: from 170 the loss is 20, not 70); balances below `minWithdraw` are unredeemable with *no* max at all, so the max only widens an existing dust class; and pTokens are transferable, so holders can pool/shed dust to a partitionable amount. "Permanently locked" needs deposits disabled *and* no counterparty. Inconsistent gating: the report dropped the strictly worse `setLimits` "maxWithdraw = 0" total freeze as admin-adversarial while promoting this one at 75.

[70] 13. `wrap` charges an execution-time fee with no caller-supplied bound

`PrivacyPortal.wrap` · Confidence: 70 · [agents: 5]

Description `wrap` is the only value entry point that derives `portalFee` from the live floor instead of validating a caller-declared value, so a fee or oracle move between quote and inclusion (market drift, or an operator `setDepositFee` front-run) is silently absorbed as protocol fee and starves the forwarded mint budget, where `deposit` would have reverted with `InsufficientPortalFee`.

Verification result — PoC: ✓ F13 · Independent verdict: **CONFIRMED** · Final severity: **Low** (confidence 85)

PoC — `findings-B.ts` > F13. User is quoted `estimateDepositFees(100) == 1.0 native` (1%) and sends `1.5 native + 1000 wei` (fee + mint budget). Operator front-runs `setDefaultDepositFee(0, 1.4%, ...)`. `deposit(..., portalFee = 1.0, ...)` reverts `InsufficientPortalFee`; `wrap(...)` with the same value succeeds with `OperationFeesPaid.portalFee == 1.4 native` and `podFee == 0.1 native + 1000 wei` — 0.4 native silently moved from the user's mint budget to protocol fees (`accumulatedPortalFees == 1.4e18`).

Independent verification (critic)

CONFIRMED · **Low** · confidence 85. Tight PoC, arithmetic checks out (1% → 1.4% of 100 USD at 1:1 pegs = 1.4 native, under the 10-native cap); `wrap` derives the floor at execution time (:498) and `_validateAndCollectPortalFee` trivially accepts it (:888-898) with the remainder becoming the mint budget (:410), whereas `deposit` validates the caller's declared fee. Precondition: an operator fee change (same semi-trusted role as #7) or an ordinary oracle move — the latter needs no privilege. Impact bounded by `maxFee` (:893-895; `PrivacyPortalFeeLib.sol`:73-74) — overpay up to the cap, or, if the squeeze pushes the mint budget under the inbox minimum, a full revert in production (`FeeManager.sol`:164-170; the mock inbox does not model minimums, so the PoC's "starved budget" reads milder than production). `wrap` is the ERC-7984 integrator path; `deposit` remains the bounded primary path. No fix proposed; add a `maxPortalFee` bound to `wrap`.

[70] 14. Escrow, burn, and withdrawal state are keyed on inbox request ids that are unique only per inbox instance

`PrivacyPortal._deposit` · Confidence: 70 · [agents: 7]

Description `depositEscrows[requestId]` and `burnInFlight[burnRequestId]` are written unconditionally and `PodERC20._setRequestStatus` does not guard writes of `Pending`, so if the admin re-points the pToken at a fresh inbox (`configurePToken`) whose per-target nonce restarts, colliding ids overwrite live escrows (destroying refund claims), double-count `burnInFlightTotal` (wedging batch burns), and can resurrect a stranded withdrawal's `transferRequestId` to `Success` — letting `triggerWithdrawalRelease` pay out collateral for a pToken transfer that never happened.

Verification result — PoC: ✓ F14 · Independent verdict: **CONFIRMED** · Final severity: **Medium** (confidence 82)

PoC — `findings-B.ts` > F14. The mock inbox reproduces the real id scheme (`InboxBase._packRequestId` , `src<<192 | target<<128 | nonce` , `InboxBase.sol:650-662`) and the real per-target nonce that restarts at 1 for a fresh instance (`++_requestNonce[targetChainId]` , `InboxBase.sol:507`). On inbox #1: registration = nonce 1, deposit D1 = nonce 2 (`Success`), withdrawal W1 transfer = nonce 3 (`Pending` , stranded). Admin `configurePToken(pToken, inbox#2, ...)` . On inbox #2, user2's deposits take nonces 1, 2, 3:

- nonce 2 == D1: `depositEscrows[D1]` now belongs to user2 (amount 10), `pToken.requests(D1).status` rewound `Success → Pending` (`_setRequestStatus` guards only terminal writes, `PodERC20.sol:628-637`);
- nonce 3 == W1's `transferRequestId`: the mint success for user2's deposit flips it to `Success` ; a stranger's `triggerWithdrawalRelease(W1)` pays 50 mUSD to W1's recipient and `pendingBurnAmount` becomes 50e6 for pTokens the portal never received on COTI.

Independent verification (critic)

CONFIRMED · **Medium** · confidence 82. Every step is anchored in production code: `_packRequestId` mixes only (source chain, target chain, nonce) with **no inbox component** (`InboxBase.sol:650-662`) and a fresh deployment restarts the per-target nonce at 1 (:507); escrow records are assigned unconditionally (`PrivacyPortal.sol:420, :468`); `_setRequestStatus` guards only terminal writes and lets `Pending` overwrite any terminal status (`PodERC20.sol:628-643`); the permissionless `triggerWithdrawalRelease` then pays real collateral for a pToken transfer that never settled. Precondition narrows the likelihood and the report should have said so: `DEFAULT_ADMIN` must `configurePToken` (`PrivacyPortalFactory.sol:375-382`) a live pToken to a **newly deployed** inbox — a proxy-style upgrade keeps address and nonce and is harmless; only a from-scratch redeploy fires it. Where it fires the impact is severe and neither self-healing nor admin-recoverable. **Unverified sub-claim:** the `burnInFlightTotal` double-count / wedged batch burns (:693-694) is asserted in the description but not exercised by the PoC — plausible from inspection, flagged as unproven.

Findings List — verification summary

#	Finding	Report conf.	PoC	Independent verdict	Final severity	Final conf.
1	Stuck withdrawal after pToken transfer Success (both legs frozen)	[90]	✓ F1a native · ✓ F1b ERC-20	CONFIRMED	Medium	92
2	Blacklist/pause not enforced on the release leg	[85]	✓ F2	CONFIRMED - DOWNGRADED	Low	85
3	Remount minter rotation bricks <code>adminRefundPendingDeposit</code>	[85]	✓ F3 (+3 recovery paths)	CONFIRMED - DOWNGRADED	Low	88
4	Cross-factory remount skips all old-portal guards	[80]	✓ F4	CONFIRMED	Medium	88
5	<code>createPortal</code> live before COTI registration	[80]	✓ F5 (incl. real mother)	DESIGN- CHOICE	Low	90
6	Remount rotates minter with no in-flight-escrow check	[80]	✓ F6	DUPLICATE	Informational	95
7	Operator freezes withdrawals via unbounded <code>fixedFee</code>	[75]	✓ F7	CONFIRMED	Low	88
8	<code>refundFailedDeposit</code> rejects <code>Failed</code>	[75]	✓ F8	CONFIRMED - DOWNGRADED	Low	85

#	Finding	Report conf.	PoC	Independent verdict	Final severity	Final conf.
9	Deposit limits/fee on requested <code>amount</code> vs measured <code>received</code>	[75]	✓ F9	CONFIRMED - DOWNGRADED	Low	82
10	<code>rescueERC20</code> drains collateral committed to pending withdrawals	[75]	✓ F10	CONFIRMED - DOWNGRADED	Low	87
11	Fee floor collapses to <code>fixedFee</code> /0 on zero oracle rate	[75]	✓ F11	DESIGN- CHOICE	Informational	90
12	<code>setLimits</code> allows an unpartitionable withdraw window	[75]	✓ F12	CONFIRMED - DOWNGRADED	Informational	80
13	<code>wrap</code> has no caller-side fee bound	[70]	✓ F13	CONFIRMED	Low	85
14	Request-id reuse across inbox rotation corrupts escrow/withdrawal state	[70]	✓ F14	CONFIRMED	Medium	82

Corrections and additional observations (rev 2)

Where the PoCs or the code contradict rev 1

- #8's core framing is wrong.** "Clearing a stale mint kills the permissionless refund" — the PoCs show `refundFailedDeposit` reverting `DepositMintNotFailed` for a *Pending* mint too (findings-A.ts:151, :254), so no permissionless refund existed to kill; the kill strictly improves the situation, and the exclusion is documented at `IPrivacyPortal.sol:155-159`.
- #3's Fix Option B cannot be implemented.** `IPrivacyPortal.pendingEscrowCount()` does not exist and is not derivable: escrows are set `Pending` (`PrivacyPortal.sol:420, :468`) and only ever cleared to `Refunded` (`:597, :638`); a mint *success* never terminalizes one, so the counter would be permanently non-zero and block every future remount.
- #1's Fix Option A only fixes half the finding.** Re-wrapping to WETH addresses the native case (F1a) but not F1b, where the ERC-20 transfer to an issuer-blocked recipient is itself the revert.
- #2 asserts as a defect what the repo tests as intended.** The pause bypass on the release leg is required by `PrivacyPortalFactory.portalRemount.test.ts:383-435` and the migration model at `PrivacyPortalFactory.sol:600-601` ; rev 1's leads section concedes it while the finding body treats it as a vulnerability.
- #6 is a self-declared duplicate of #3**, and **#12 contradicts rev 1's own gating note**, which dropped the strictly worse `setLimits` "maxWithdraw = 0" freeze as admin-adversarial. Both inflated the finding count.

Material observations surfaced during verification (not deep-dived)

- `feeRecipient` is `immutable` with no setter** (`PrivacyPortalFactory.sol:52, 205`) while `withdrawPortalFees` hard-requires the send to succeed (`PrivacyPortal.sol:742-743`). If that address ever stops accepting ETH, every portal's accumulated fees are stranded factory-wide; the only escape is `rescueNative` , which pays the (mutable) `rescueRecipient` instead.
- `setLimits` accepts `maxWithdraw == 0` / `maxDeposit == 0`** — only `min > max` is rejected (`PrivacyPortal.sol:342-347`) — a silent total freeze with no `Paused` event, i.e. #7's shape one privilege level up. Rev 1 discarded this while promoting the weaker #12.

3. **No on-chain record exists of pTokens the portal actually holds on COTI.** `pendingBurnAmount` is incremented *inside* `_releaseWithdrawal` (:866), so a reverting payout rolls the accounting back. This one ordering choice is the shared root cause of #1's unburnable custody and #10's invisible obligation and deserves to be stated as its own invariant break.
4. **PoC-fidelity caveat.** `MockInboxForPortal` enforces no fee minimums, whereas the production `FeeManager` reverts `TotalFeeTooLow` / `CallbackFeeTooLow` (`coti-pod-inbox-contracts/contracts/fee/FeeManager.sol:164-170`). Consequences: #13's "starved mint budget" is a transaction revert in production rather than an under-funded message, and the fixture's `createPortal` with `msg.value == 0` would not be accepted on a real chain. No finding's mechanism depends on this.
5. **The "1-day kill delay" that #3 leans on is a one-transaction admin knob.** `setPTokenRequestKillMinAge` (`PrivacyPortalFactory.sol:400-403` → `PodERC20.sol:584-586`) lets `DEFAULT_ADMIN` set the age to 0 and kill immediately (PoC recovery path C).

Leads

Vulnerability trails with concrete code smells where the full exploit path could not be completed in one analysis pass. These are not false positives — they are high-signal leads for manual review. Not scored.

- **Permissionless release entry inherits the release-leg gap** — `PrivacyPortal.triggerWithdrawalRelease` — Code smells: no access control, no pause/blacklist check — the surface through which #2 fires and through which a resurrected `transferRequestId` (#14) would be cashed; the pause-bypass is asserted as intended by a remount test, which is what creates the #10 migration race.
- **burnInFlight overwrite double-counts burnInFlightTotal** — `PrivacyPortal.burnAccumulatedPTokens` — Code smells: `burnInFlight[id] = amount` (assignment) beside `burnInFlightTotal += amount` (increment) with no occupancy check — on an id collision (#14) the total inflates permanently and `available` underflows, wedging batch burns.
- **Forgeable OnlyPToken guard** — `PrivacyPortal.onPTokenTransferred` — Code smells: `transferAndCall` lets any holder make the pToken issue arbitrary calldata with `msg.sender == pToken` — harmless today only because the handler re-validates everything; any future pToken-trusting handler on the portal would be directly exploitable.
- **Unguarded oracle call on every user path, including exit** — `PrivacyPortal._portalFeeFloor` / `requestWithdrawWithPermit` — Code smells: `oracle.getLivePrices` with no try/catch; `IPodPriceOracle` only *promises* non-revert and `setPriceOracle` checks only code length — a reverting or gas-bombing adapter bricks deposits and new withdrawals factory-wide; no shipped adapter reproduces it.
- **Force- Failed transfer may close a withdrawal whose COTI leg settled** — `PrivacyPortal.cancelFailedWithdrawal` — Code smells: `killStaleRequest` / `invalidatePendingRequest` write `Failed` with no knowledge of COTI settlement — if the leg did settle, pTokens are custodied uncounted and the ticket closes with no collateral released; unverified whether COTI settlement can outlive a killed request.
- **pendingBurnAmount can be permanently over-stated** — `PrivacyPortal.finalizeBatchBurn` — Code smells: only `Success` decrements it; a burn that settled on COTI but lost its callback, then admin-killed to `Failed`, takes the failure branch with no corrective setter.
- **Outbound legs are unmeasured; refunds don't unwrap** — `PrivacyPortal._releaseWithdrawal` / `refundFailedDeposit` — Code smells: unwrap has no native-delta check (a shortfall would be paid from the fee balance); outbound fee-on-transfer direction untested (the mock's "portal never calls `transfer`" comment is false); `depositNative` users are refunded in WETH.

- **18-decimal assumption for the wrapped native** — `PrivacyPortal.depositNative` — Code smells: `amount` used both as wei and as token units for limits/fee-notional; the factory never asserts `decimals == 18` for the native-wrapped case; no deployment target found where it breaks.
- **Per-portal risk controls reset on remount** — `PrivacyPortal.initialize` — Code smells: the new clone starts with an empty per-portal blacklist, default limits, cleared fee overrides; only factory-wide controls survive; window exists only if the admin unpauses without reconfiguring.
- **One-shot, unacknowledged mother registration; underlying slot can be squatted** — `PrivacyPortalFactory.createPortal` — Code smells: fire-and-forget `sendOneWayMessage` with no error selector, `_requestMotherRegistration` private with no re-send, mappings never cleared, `createPortal` is the lowest-trust `DEPLOYER_ROLE` — a failed registration or a deployer pre-emption permanently occupies the underlying; unverified whether inbox retry re-drives the message.
- **Rotating the COTI peer on a live pToken strands in-flight requests** — `PrivacyPortalFactory.configurePToken` — Code smells: overwrites `trustedRemote`, so old-peer callbacks fail `UntrustedPeer` and requests never leave `Pending`; no paused precondition, unlike the remount path.
- **setPTokenMinter leaves the mapped portal advertising itself live** — `PrivacyPortalFactory.setPTokenMinter` — Code smells: no coupled retire/pause/mapping update; deposits revert `OnlyMinter` while `isDepositEnabled` / `paused()` still read open.
- **Constructor skips the code-existence checks the setters enforce** — `PrivacyPortalFactory.constructor` — Code smells: implementations/oracle only zero-checked at deploy; a codeless `portalImplementation` yields silently "initialized" clones that permanently occupy `portalForUnderlying` (no delete path), a codeless oracle bricks every portal's fee path.

 This review was performed by an AI assistant. AI analysis can never verify the complete absence of vulnerabilities and no guarantee of security is given. Team security reviews, bug bounty programs, and on-chain monitoring are strongly recommended. For a consultation regarding your projects' security, visit <https://www.pashov.com>